

Bit, Byte ve Integer

BIL-304: Bilgisayar Mimarisi

Dersi veren öğretim üyesi:

Dr. Öğr. Üyesi Dr. Fatih Gökçe

Ders kitabına ait sunum dosyalarından adapte edilmiştir: <http://csapp.cs.cmu.edu/>

Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- Bit düzeyinde manipülasyonlar
- **Integer**
 - unsigned ve signed
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - **Genişletme (expanding), kırpma (truncating)**
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)

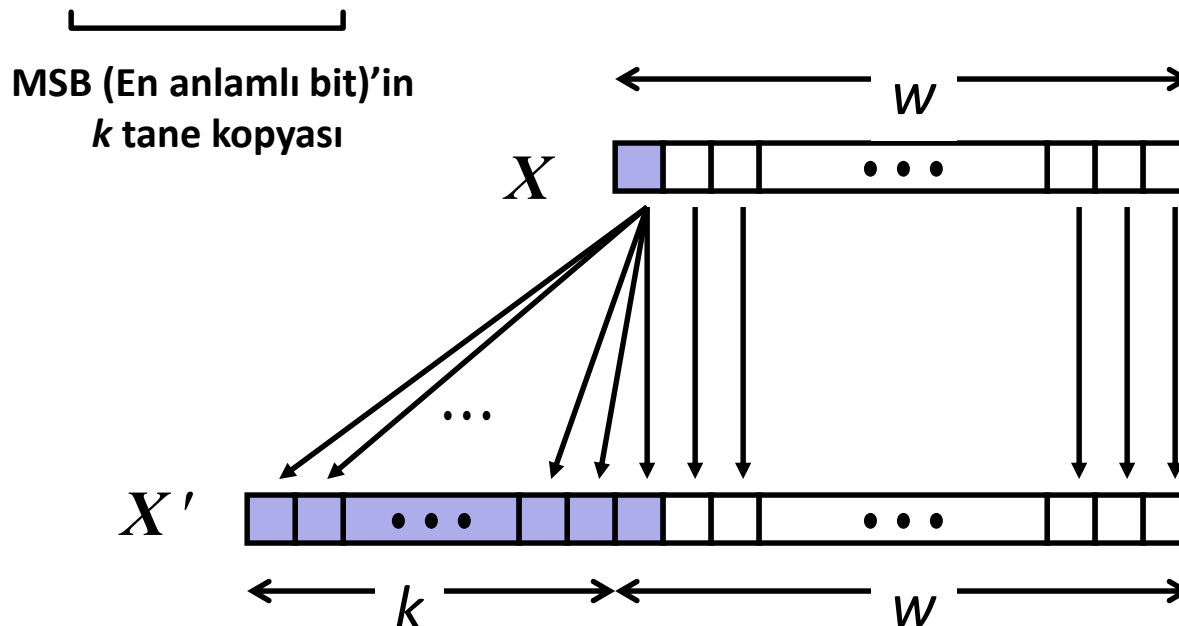
İşaret Genişletme (Extension)

■ Görev:

- w -bit boyutunda signed integer x veriliyor.
- x 'i $w+k$ -bitlik aynı değerdeki integer'a dönüştür.

■ Kural:

- İşaret bitinin k tane kopyasını oluştur:
- $X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ tane kopyası}}, x_{w-1}, x_{w-2}, \dots, x_0$



İşaret Genişletme Örnek

```
short int x = 15213;
int      ix = (int) x;
short int y = -15213;
int      iy = (int) y;
```

	Onluk	Hex	İkili
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011

- 16 bitlik küçük integer veri tipinden (short int) 32 bitlik büyük integer veri tipine (int) dönüştürme
- C otomatik olarak işaret genişletme uygular

Özet:

Genişletme(Expanding), Kırpma (Truncating): Temel Kurallar

- **Genişletme (örneğin, short int'ten int'e)**
 - Unsigned: Sıfırlar eklenir
 - Signed: İşaret genişletme
 - Her ikisi de beklenen sonuçları verir.

- **Kırpma (örneğin, unsigned'dan unsigned short'a)**
 - Unsigned/signed: bitler kırılır.
 - Sonuç tekrar yorumlanır
 - Unsigned: mod alma işlemine eşdeğerdir.
 - Signed: mod alma işlemine benzer.
 - Küçük sayılar için beklenen sonuçları verir.

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- Bit düzeyinde manipülasyonlar
- **Integer**
 - unsigned ve signed
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - Genişletme (expanding), kırpma (truncating)
 - **Toplama, işaret değiştirme (negation), çarpma, kaydırma**
- Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)

Unsigned Toplama

u ve v operandları: w bitlik



Gerçek Toplam: $w+1$ bit



Elde ihmal edilir: w bit



■ Standart Toplama Fonksiyonu

- Elde çıktısını ihmal eder.

■ Modüler Aritmetiği Uygular

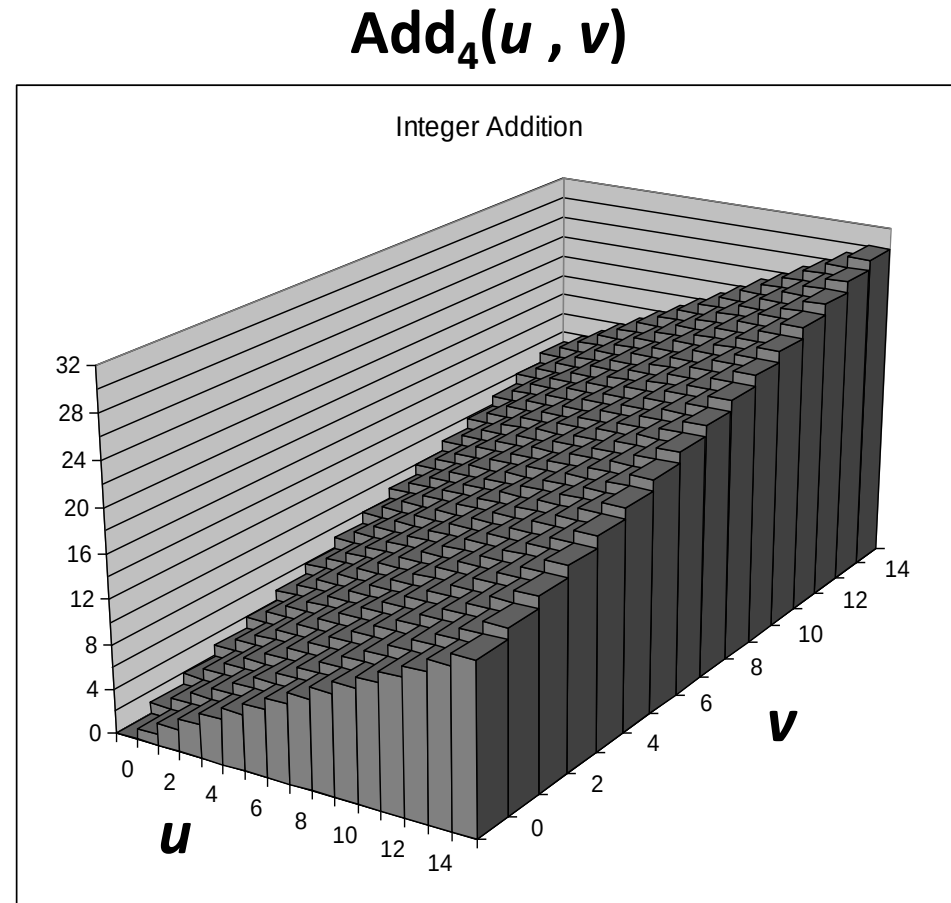
$$s = \text{UAdd}_w(u, v) = u + v \bmod 2^w$$

$$\text{UAdd}_w(u, v) = \begin{cases} u + v & u + v < 2^w \\ u + v - 2^w & u + v \geq 2^w \end{cases}$$

(Matematiksel) Integer Toplamanın Görselleştirilmesi

■ Integer Toplama

- 4-bitlik integerlar u , v
- Gerçek toplamı hesaplar $Add_4(u, v)$
- Toplam değer u ve v ile doğrusal şekilde artar.
- Düzlemsel bir yüzey oluşturur.

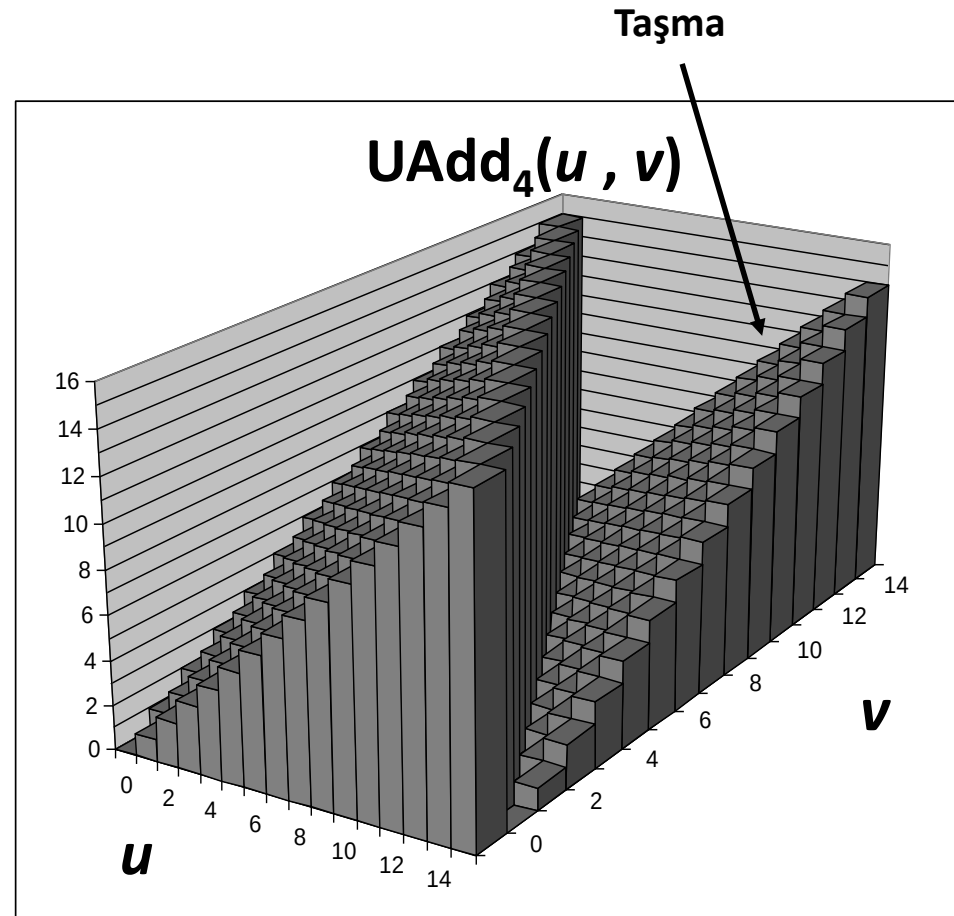
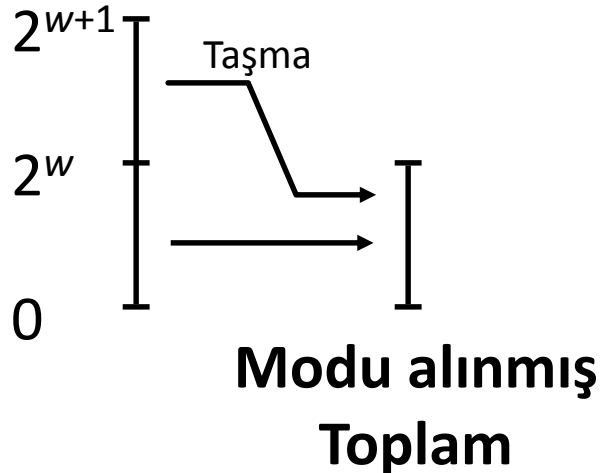


Unsigned Toplamanın Görselleştirilmesi

■ Başa sarar

- Eğer gerçek toplam $\geq 2^w$ ise
- En fazla bir kez başa sarar

Gerçek Toplam

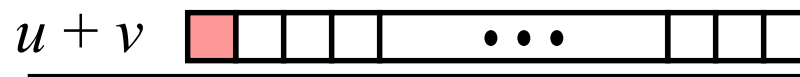


2 Tümleyen ile ifade edilen sayıların Toplamı

u ve v operandları: w bitlik



Gerçek Toplam: $w+1$ bit



Elde ihmal edilir: w bit



■ TAdd ve UAdd Bit-seviyesinde bakarsak birebir aynı işlemi yapar:

- C'de Signed ve unsigned toplama karşılaştırılırsa

```
int s, t, u, v;
```

```
s = (int) ((unsigned) u + (unsigned) v);
```

```
t = u + v
```

- $s == t$ doğru döner.

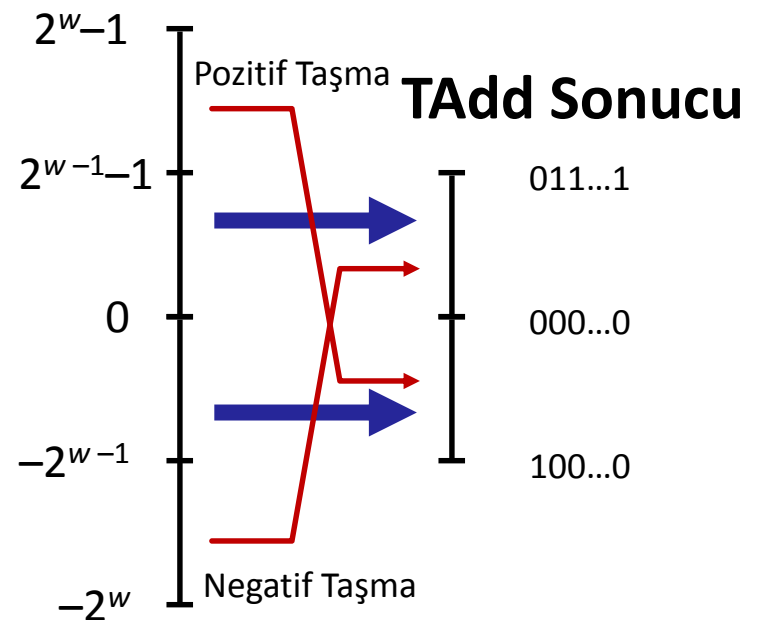
TAdd Taşması (Elde biti taşmasından farklıdır.)

■ Toplamanın İşlevi

- Gerçek toplam $w+1$ bit gerektirir.
- MSB, yani en anlamlı bit düşürülür, atılır.
- Geriye kalan bit dizisi 2 Tümleyene göre oluşturulmuş bir integer olarak değerlendirilir.

0 111...1
0 100...0
0 000...0
1 011...1
1 000...0

Gerçek Toplam



2 Tümleyen Toplamasının Görselleştirilmesi

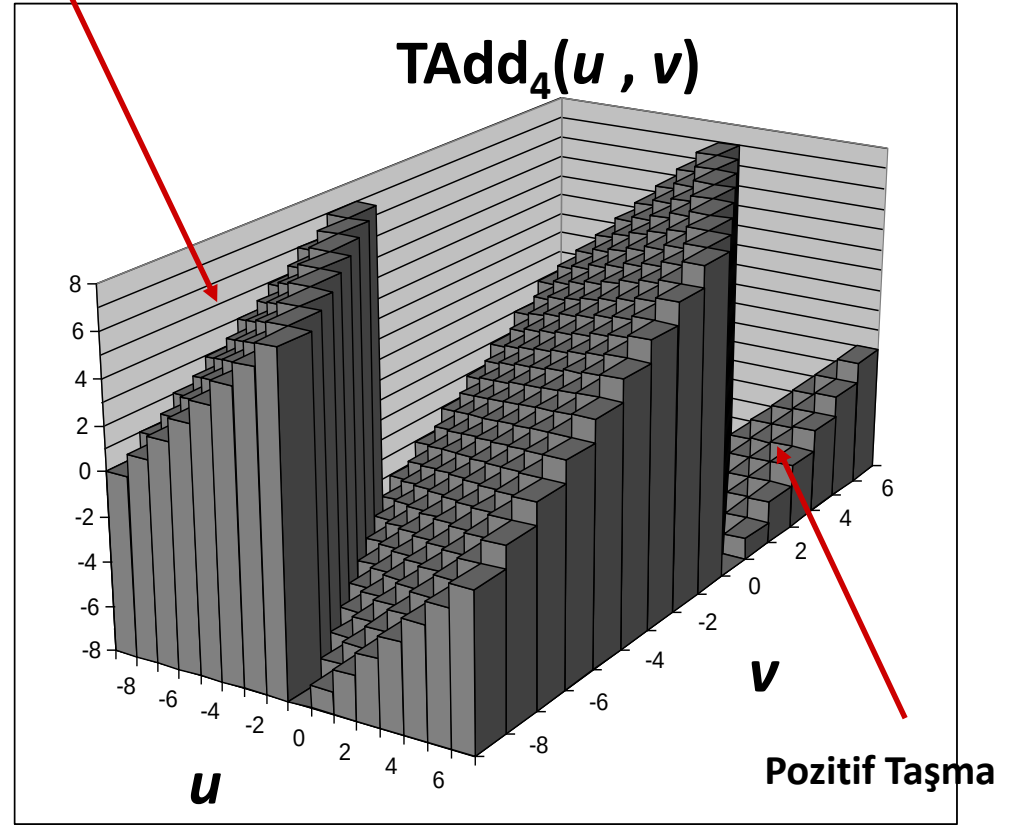
■ Değerler

- 4-bit 2 Tümleyen sayılar.
- -8 to +7 aralığındadırlar.

■ Başa sarar

- Eğer toplam $\geq 2^{w-1}$
 - Negatif olur.
 - En fazla bir kez başa sarar.
- Eğer toplam $< -2^{w-1}$
 - Pozitif olur
 - En fazla bir kez başa sarar.

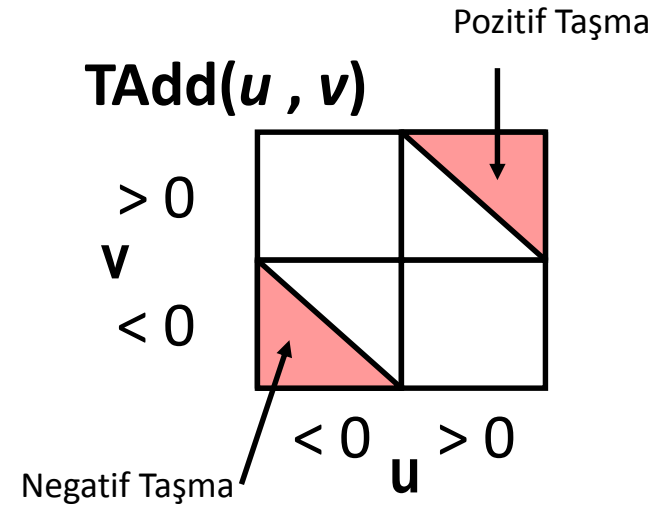
Negatif Taşma



TAdd'ın tanımlanması

■ İşlevi

- Gerçek toplam $w+1$ bit gerektirir
- MSB, yani en anlamlı bit düşürülür, atılır.
- Geriye kalan bit dizisi 2 Tümleyene göre oluşturulmuş bir integer olarak değerlendirilir.



$$TAdd_w(u, v) = \begin{cases} u + v + 2^w & u + v < TMin_w \quad (\text{Negatif Taşma}) \\ u + v & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w & TMax_w < u + v \quad (\text{Pozitif Taşma}) \end{cases}$$

TAdd'ın tanımlanması

- İki pozitif sayının toplamı negatif ise pozitif taşma; iki negatif sayının toplamı pozitif ise negatif taşma olmuş demektir.
- Bit düzeyinde toplama işleminde (w+1). bite aktarılması gereken elde oluşmuş olması pozitif veya negatif taşmayı garantilemez.
- (w+1). bite aktarılması gereken elde oluşmadan da taşma oluşabilir; elde oluşup taşmanın olmaması da mümkündür.
- <http://sandbox.mc.edu/~bennet/cs110/tc/add.html> den alınan şu örnekleri inceleyiniz.

• $-39 + 92 = 53$:

$$\begin{array}{r} \boxed{1} \quad 1 \quad 1 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \\ + \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \\ \hline 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \end{array}$$

Carryout without overflow. Sum is correct.

• $-19 + -7 = -26$:

$$\begin{array}{r} \boxed{1} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \\ + \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \\ \hline 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \end{array}$$

Carryout without overflow. Sum is correct.

• $44 + 45 = 89$:

$$\begin{array}{r} \quad 1 \quad 1 \quad 1 \\ 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \\ + \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \\ \hline 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \end{array}$$

No overflow nor carryout.

• $104 + 45 = 149$:

$$\begin{array}{r} \quad 1 \quad 1 \quad 1 \\ 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \\ + \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \\ \hline 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \end{array}$$

Overflow, no carryout. Sum is not correct.

• $-75 + 59 = -16$:

$$\begin{array}{r} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \\ + \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \\ \hline 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}$$

No overflow nor carryout.

• $-103 + -69 = -172$:

$$\begin{array}{r} \boxed{1} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \\ + \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \\ \hline 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \end{array}$$

Overflow, with incidental carryout. Sum is not correct.

• $10 + -3 = 7$:

$$\begin{array}{r} \boxed{1} \quad 1 \quad 1 \quad 1 \quad 1 \\ 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \\ + \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \\ \hline 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \end{array}$$

Carryout without overflow. Sum is correct.

• $127 + 1 = 128$:

$$\begin{array}{r} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ + \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \\ \hline 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}$$

Overflow, no carryout. Sum is not correct.

• $-1 + 1 = 0$:

$$\begin{array}{r} \boxed{1} \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \\ + \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \\ \hline 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}$$

Carryout without overflow. Sum is correct.

İşaret Değiştirme (Negation): 1 Tümleyen alma + 1 Arttırma

- 2 tümleyen için şu geçerlidir:

$$\sim x + 1 == -x$$

- Gözlem: $\sim x + x == 1111\dots111 == -1$

$$\begin{array}{r}
 x \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{0} \boxed{1} \\
 + \quad \sim x \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{0} \\
 \hline
 -1 \quad \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1}
 \end{array}$$

$$Tneg_w(u) = \begin{cases} -u & u \neq TMin_w \\ TMin_w & u = TMin_w \end{cases}$$

Tmin'in işaretini değiştirmek
istediğimizde yine kendisini elde ederiz.

1 Tümlleyen alma + 1 Arttırma Örnekleri

x = 15213

	Onluk	Hex	İkili
x	15213	3B 6D	00111011 01101101
~x	-15214	C4 92	11000100 10010010
~x+1	-15213	C4 93	11000100 10010011
y	-15213	C4 93	11000100 10010011

x = 0

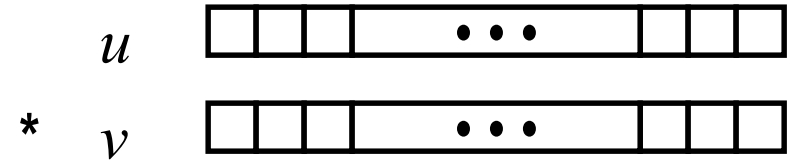
	Onluk	Hex	İkili
0	0	00 00	00000000 00000000
~0	-1	FF FF	11111111 11111111
~0+1	0	00 00	00000000 00000000

Çarpma

- **Amaç: w -bitlik x, y sayılarının çarpımını hesaplamak.**
 - signed veya unsigned
- **Fakat, kesin, gerçek sonuç w bitten büyük olabilir.**
 - Unsigned: $2w$ bite kadar
 - Sonuç aralığı: $0 \leq x * y \leq (2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$
 - 2 Tümleyen minimum (negatif): $2w-1$ bite kadar
 - Sonuç aralığı: $x * y \geq (-2^{w-1})*(2^{w-1}-1) = -2^{2w-2} + 2^{w-1}$
 - 2 Tümleyen maksimum (pozitif): $2w$ bite kadar, fakat sadece $(TMin_w)^2$ için
 - Sonuç aralığı: $x * y \leq (-2^{w-1})^2 = 2^{2w-2}$
- **Bu nedenle, gerçek sonucu koruyabilmek için...**
 - Word size' in her bir çarpımda arttırılmasını gerektirir.
 - Yazılımsal olarak gerçekleştirilir. Gerek duyulmuşsa
 - örneğin, “arbitrary precision” (isteğe bağlı hassasiyet sağlayan) aritmetik paketleri kullanılabilir.

C'de Unsigned Çarpma

u ve v operandları: w bitlik



*



$u \cdot v$

Gerçek Çarpım: $2 \cdot w$ bit

w bitlik soldaki kısım ihmal edilir: w bit kalır. $\text{UMult}_w(u, v)$



■ Standart Çarpma Fonksiyonu

- Yüksek öncelikli w bit ihmal edilir.

■ Modüler Aritmetiği Uygular

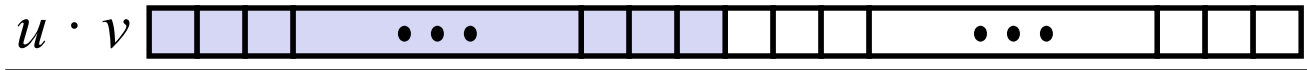
$$\text{UMult}_w(u, v) = u \cdot v \bmod 2^w$$

C'de Signed (İşaretli) Çarpma

u ve v operandları: w bitlik



Gerçek Çarpım : $2*w$ bit



w bitlik soldaki kısım ihmal edilir: w bit kalır. $\text{TMult}_w(u, v)$

■ Standart Çarpma Fonksiyonu

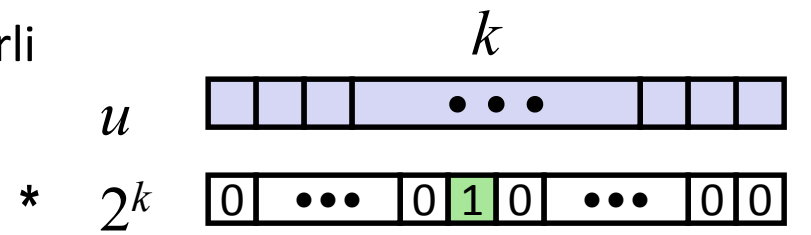
- Yüksek öncelikli w bit ihmal edilir.
- Kalan w bitlik kısım işaretli olarak yorumlandığı için işaretsiz (unsigned) çarpmaya göre farklılık oluşabilir. Fakat bit düzeyinde işaretli ve işaretsiz çarpmada aynı işlemleri yürütülür. Bu nedenle ihmal edilen kısım haricinde kalan düşük öncelikli kısımdaki bitler aynıdır.

Kaydırma (Shift) ile 2'nin kuvveti çarpımı

■ İşlem

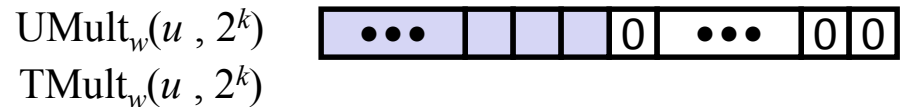
- $u \ll k$, $u * 2^k$ 'yı verir.
- signed ve unsigned ikisi için de geçerli

Operandlar: w bitlik



Gerçek çarpım: $w+k$ bit $u \cdot 2^k$

k bitlik soldaki kısım ihmal edilir: w bit kalır.



■ Örnekler

- $u \ll 3 == u * 8$
- $(u \ll 5) - (u \ll 3) == u * 24$
- Bir çok bilgisayar kaydırma işlemini çarpmadan daha hızlı yapar.
 - Derleyici bu kodu otomatik olarak oluşturur.

Derlenmiş Çarpma Kodu

C Fonksiyonu

```
long mul12(long x)
{
    return x*12;
}
```

Derlenmiş Aritmetik İşlemler

```
leaq (%rax,%rax,2), %rax
salq $2, %rax
```

Açıklama

```
t <- x+x*2
return t << 2;
```

- C derleyicisi sabit bir sayı ile çarpma durumunda kaydırma/toplama içeren kodu otomatik olarak oluşturur.

Aritmetik: Temel Kurallar

■ Toplama:

- Unsigned/signed: Normal toplama yapılıp ardından kırpma (truncate) yapılır,
bit düzeyinde aynı işlemler yapılır

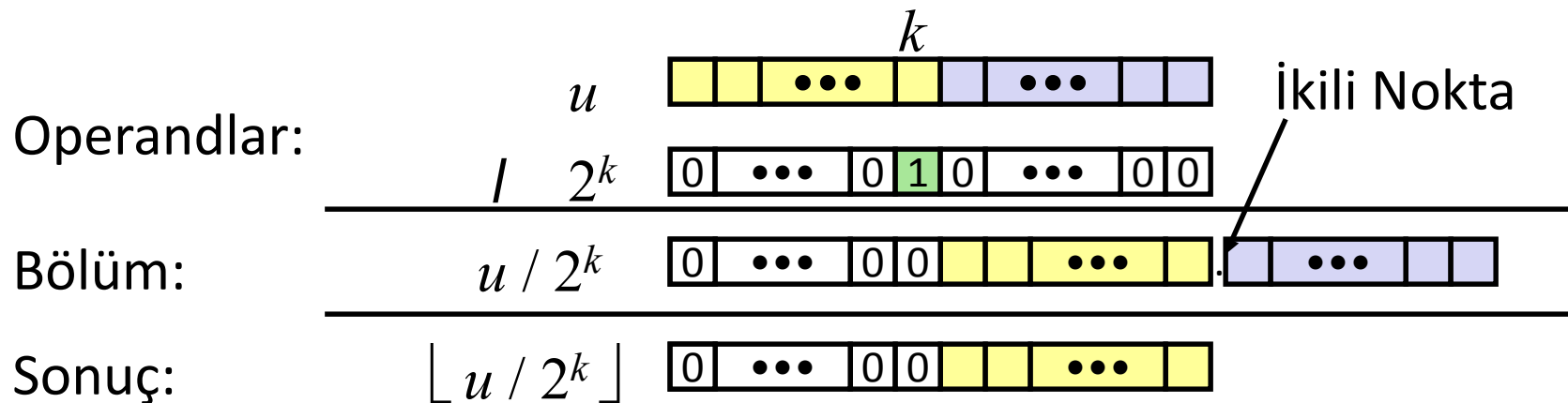
■ Çarpma:

- Unsigned/signed: Normal çarpma yapılıp ardından kırpma (truncate) yapılır,
bit düzeyinde aynı işlemler yapılır

Kaydırma ile Unsigned 2'nin kuvvetine bölme

■ Unsigned (işaretsiz) sayının 2'nin kuvvetine bölümü

- $u \gg k$, $\lfloor u / 2^k \rfloor$ 'i verir
- Lojik kaydırma kullanılır.



	Bölüm	Hesaplanan	Hex	İkili
x	15213	15213	3B 6D	00111011 01101101
x >> 1	7606.5	7606	1D B6	00011101 10110110
x >> 4	950.8125	950	03 B6	00000011 10110110
x >> 8	59.4257813	59	00 3B	00000000 00111011

Derlenmiş Unsigned Bölme Kodu

C Fonksiyonu

```
unsigned long udiv8  
    (unsigned long x)  
{  
    return x/8;  
}
```

Derlenmiş Aritmetic İşlemler

```
shrq $3, %rax
```

Açıklama

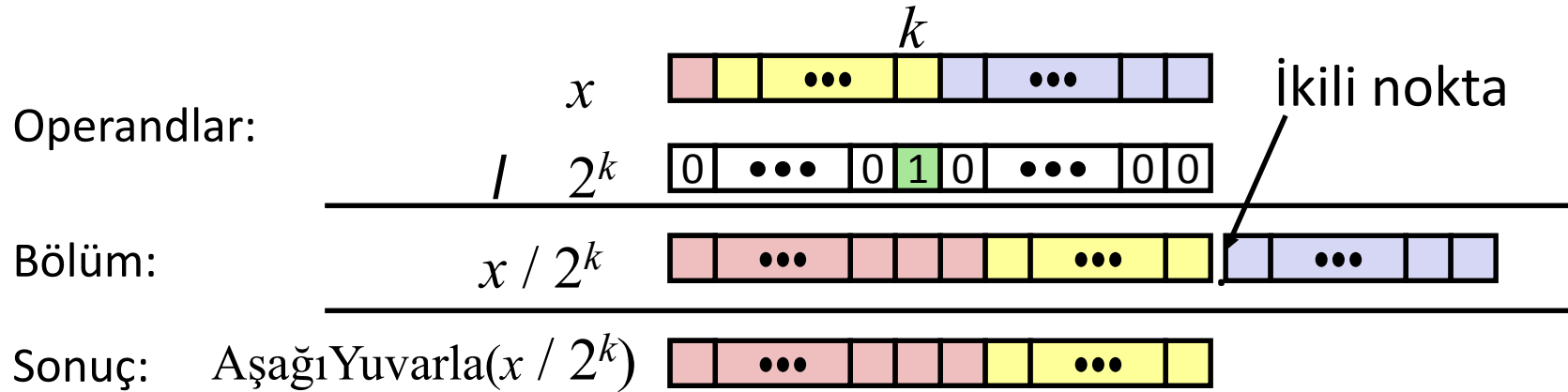
```
# Lojik kaydırma  
return x >> 3;
```

- Unsigned bölmede lojik kaydırma kullanılır.
- Java kullanıcıları için
 - Lojik kaydırma >>> ile gösterilir.

Kaydırma ile Signed 2'nin Kuvvetine Bölme

■ Signed (işaretli) sayının 2'nin kuvvetine bölümü

- $x \gg k, \lfloor x / 2^k \rfloor$ 'yi verir.
- Aritmetik kaydırma kullanılır.
- $u < 0$ iken yanlış tarafa yuvarlama söz konusudur (Çözülecek).



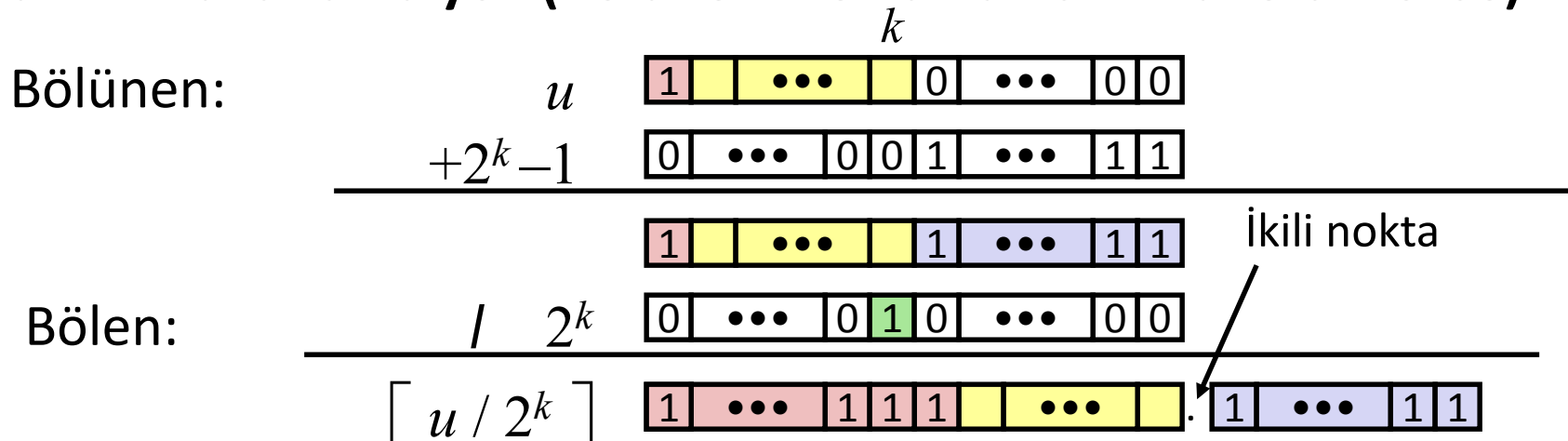
	Bölüm	Hesaplanan	Hex	İkili
y	-15213	-15213	C4 93	11000100 10010011
y >> 1	-7606.5	-7607	E2 49	1 1100010 01001001
y >> 4	-950.8125	-951	FC 49	1111 1100 01001001
y >> 8	-59.4257813	-60	FF C4	11111111 11000100

Düzeltilmiş 2'nin Kuvvetine Bölme

■ Negatif sayının 2'nin kuvvetine bölümü

- İstenen: $\lceil x / 2^k \rceil$ (0'a doğru yuvarlama)
- $\lfloor (x+2^k-1) / 2^k \rfloor$ şeklinde hesaplanır.
 - C'de: $(x + (1 \ll k) - 1) \gg k$
 - Bölünen 0'a doğru yönlendirilmiş olur.

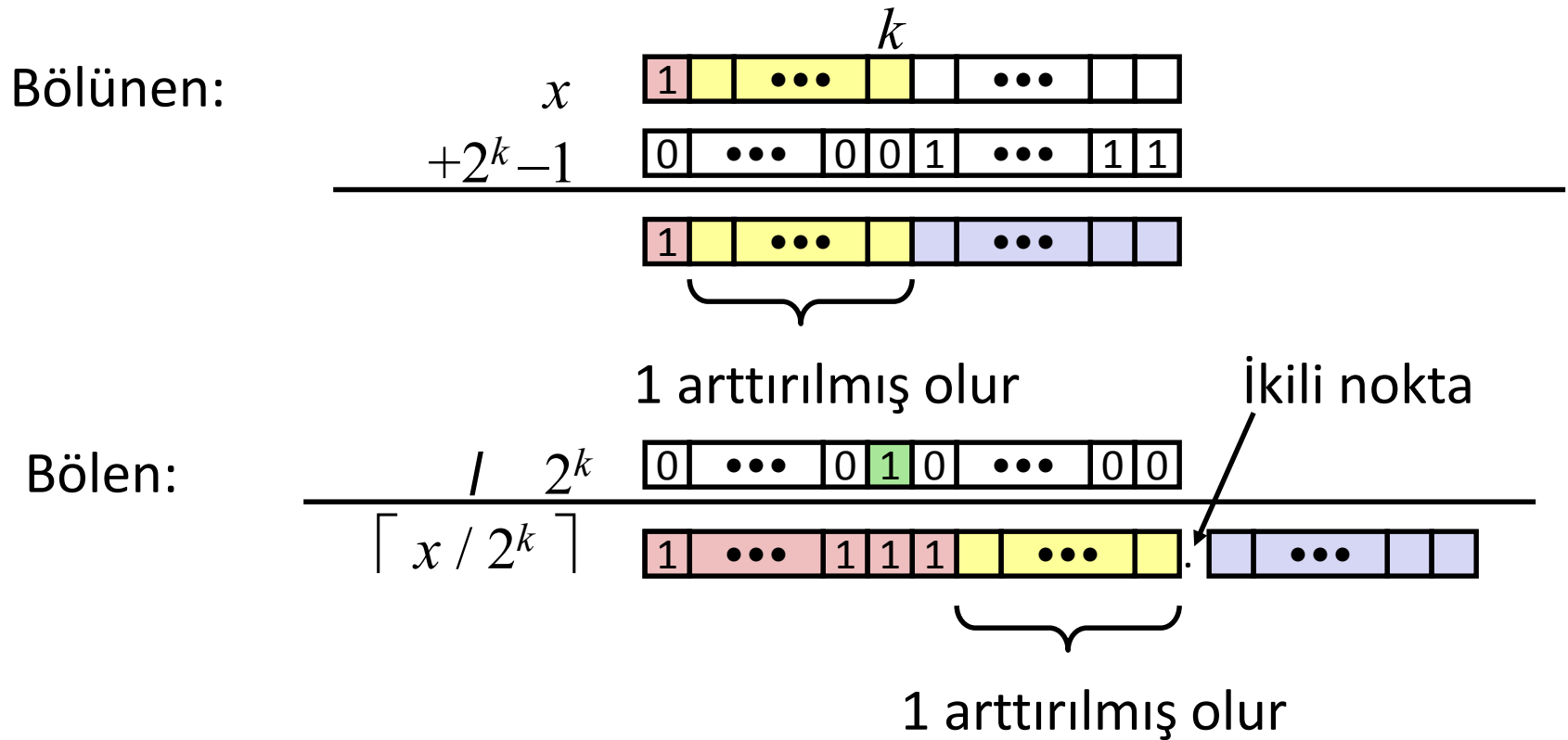
Durum 1: Yuvarlama yok (Bölünenin en anlamsız k tane biti 0 ise)



Biasing (yönlendirme)'in etkisi yoktur.

Düzeltilmiş 2'nin Kuvvetine Bölme

Durum 2: Yuvarlama



Biasing (yönlendirme) en son sonuca 1 ekler.

Derlenmiş Signed (işaretli) Bölme Kodu

C Fonksiyonu

```
long idiv8(long x)
{
    return x/8;
}
```

Derlenmiş Aritmetik İşlemler

```
testq %rax, %rax
js    L4
L3:
    sarq $3, %rax
    ret
L4:
    addq $7, %rax
    jmp  L3
```

Açıklama

```
if x < 0
    x += 7;
# Aritmetik kaydırma
return x >> 3;
```

- int için aritmetik kaydırma kullanılır
- Java kullanıcıları için:
 - Aritmetik kaydırma >> ile gösterilir.